PROJECT PROPOSAL

E-commerce Supply Chain Data Engineering with a Self-Healing Data Pipeline

Status: In Progress | Target completion: 4 weeks

Proposal draft for the DEPI team. This document describes the intended solution, not completed results. Confirm final roles, scope, and evaluation thresholds with the team and mentor.

1. Executive Summary

This project proposes an end-to-end data engineering workflow for e-commerce supply-chain data. The workflow will process the selected dataset, validate data quality, load analysis-ready data into PostgreSQL, and use Apache Airflow to orchestrate pipeline tasks. A self-healing layer will detect common data issues and apply safe, predefined recovery actions when possible. Uncertain or unrepairable issues will be logged and flagged for review rather than silently changed. Power BI will present selected supply-chain metrics and, where feasible, data-quality indicators.

2. Problem Statement

E-commerce supply-chain datasets can contain missing values, duplicate records, inconsistent formats, invalid values, or unexpected schema changes. If these issues remain undetected, they can reduce the reliability of downstream analysis and dashboards. Manual correction is also difficult to repeat consistently. This project addresses these challenges through a testable pipeline with explicit quality checks, controlled recovery rules, and traceable exception handling.

3. Aim and Objectives

- Build a repeatable ingestion and transformation pipeline for the selected dataset.
- Profile the dataset and define data-quality rules based on its actual fields and business meaning.
- Detect applicable issues such as missing required values, duplicates, invalid ranges, inconsistent formats, and schema deviations.
- Implement safe self-healing rules for selected known issues, with logs and validation after recovery.
- Store validated data in PostgreSQL and orchestrate tasks with Apache Airflow.
- Use Pytest to test transformations, validation checks, and recovery behavior.
- Create a Power BI dashboard for relevant supply-chain KPIs and, if feasible, pipeline/data-quality summaries.

4. Proposed Workflow

1. Data source: use the dataset already selected by the team and document its source, license, schema, and assumptions.

2. Ingestion and profiling: load source data and inspect data types, missing values, duplicates, distributions, and key-field consistency.

3. Transformation: use Python and Pandas to standardize formats and prepare data for storage.

4. Validation: run documented quality checks, such as required-field, uniqueness, valid-range, and schema checks where relevant.

5. Self-healing: apply only explicit, safe rules to known issues. Examples may include format normalization, duplicate removal under a documented key, and retrying transient task failures. Log every action and validate the result.

6. Exception handling: quarantine or flag ambiguous cases for human review; do not invent values or silently overwrite uncertain data.

7. Storage and orchestration: load validated data into PostgreSQL and coordinate pipeline stages through an Apache Airflow DAG.

8. Testing and reporting: use Pytest and logs to demonstrate behavior; use Power BI for selected business and data-quality insights.

5. Project Scope

In scope: one selected dataset; Python/Pandas processing; PostgreSQL storage; Apache Airflow orchestration; documented data-quality checks; a limited, demonstrable self-healing layer; exception logging; Pytest coverage; and a Power BI dashboard.

Out of scope or conditional: a production-grade autonomous AI repair system, automatic changes to ambiguous business-critical values, Spark/distributed processing unless justified, a custom web app/backend unless agreed, and claims of business improvement that are not measured.

6. Proposed Technology Stack

- Python and Pandas — data profiling, transformations, validation, and recovery rules.
- SQL and PostgreSQL — querying, relational storage, and repeatable data loading.
- Apache Airflow — scheduling, task dependencies, orchestration, retries, and task status.
- Pytest — automated tests for core transformations and recovery behavior.
- Power BI — supply-chain analytics and data-quality visualization.
- Docker — optional, if time and the team environment permit.

7. Expected Deliverables

- Project proposal, requirements, planning document, and literature review.
- Dataset profile and documented data-quality rules.
- Python/Pandas pipeline modules and required SQL scripts.
- PostgreSQL schema and repeatable loading process.
- Apache Airflow DAG demonstrating the main workflow.
- A self-healing demonstration for a small number of safe, predefined issue types, including logs and post-recovery validation.
- Pytest tests for core transformations and recovery rules.
- Power BI dashboard, README/setup instructions, and final presentation/demo.

8. Four-Week Timeline

Week 1 — confirm business questions, profile the dataset, finalize requirements and quality rules, agree responsibilities, and set up the repository.

Week 2 — implement core transformations, PostgreSQL schema/loading, initial Airflow DAG, and baseline validation checks.

Week 3 — implement selected self-healing rules and exception logging, integrate stages, write tests, and build the first dashboard version.

Week 4 — run end-to-end tests, fix defects, document results and limitations, finalize the dashboard, and prepare the presentation.

9. Draft Team Responsibilities

These assignments are proposals for discussion only; they are not confirmed until the team agrees.

Menna Hamdy — coordination, requirements/documentation, and integration tracking (proposed).

Omar Salem — dataset profiling and preparation (proposed).

Ibrahim Elglad — Python/Pandas transformation modules (proposed).

Belal Yasser — SQL and PostgreSQL schema/loading (proposed).

Ahmed Sami — Apache Airflow orchestration and task monitoring (proposed).

Abdalla Elhelaly — Power BI dashboard, integration support, and presentation (proposed).

Self-healing rules, testing, and end-to-end integration should be shared responsibilities, with one clear owner for each task.

10. Success Criteria

The pipeline runs end-to-end and loads expected outputs into PostgreSQL.

Documented checks detect deliberately introduced test defects.

At least two or three safe self-healing cases are demonstrated with before/after validation and an audit log.

Uncertain or unrepairable cases are flagged or quarantined rather than silently changed.

Airflow dependencies and retry/failure behavior are demonstrated.

Core Pytest tests pass and dashboard figures can be traced to processed database tables.

Do not report error-reduction percentages, accuracy, or time savings unless measured during evaluation.

11. Risks and Mitigation

Scope growth: prioritize a working pipeline and a few reliable recovery rules before optional features.

Unclear source data: profile early and document assumptions; confirm ambiguous rules with the mentor/team.

Unsafe automatic correction: use explicit rules, preserve original values where practical, log actions, and route uncertain cases for review.

Environment/setup delays: establish a minimal working Airflow setup early and document the demo environment.

Integration bottlenecks: agree interfaces, naming conventions, task owners, and Git workflow early; integrate continuously.

Limited dashboard time: build a small useful dashboard early, then add optional metrics only after the pipeline is tested.

12. Assumptions and Dependencies

The dataset has been selected/prepared; final source details and schema will be documented in the repository.

Approximately four weeks remain, so the implementation must be prioritized by feasibility.

The exact KPIs and quality rules depend on the fields available in the dataset.

Final responsibilities, thresholds, and any DEPI-specific template requirements remain subject to confirmation.

13. Conclusion

The proposed solution combines data engineering fundamentals with a practical self-healing capability. It aims to create a repeatable and observable workflow that detects common data issues, safely recovers from selected known problems, and exposes unresolved issues. The project will focus on a feasible end-to-end demonstration using Python, Pandas, SQL, PostgreSQL, Apache Airflow, Pytest, and Power BI.

References and Further Reading

Apache Airflow Documentation: https://airflow.apache.org/docs/

PostgreSQL Documentation: https://www.postgresql.org/docs/

pandas Documentation: https://pandas.pydata.org/docs/

pytest Documentation: https://docs.pytest.org/

Microsoft Power BI Documentation: https://learn.microsoft.com/power-bi/

Kimball, R., & Caserta, J. (2004). The Data Warehouse ETL Toolkit. Wiley.

Toorajipour, R. et al. (2021). Artificial intelligence in supply chain management: A systematic literature review. Journal of Business Research, 122, 502–517.